

Lecture 13: Natural Language Generation

Instructor: Stephanie Schoch

CSCI 680: Natural Language Processing
Department of Computer Science, College of William & Mary
October 5, 2026



Lecture Outline

- Recap
- Natural Language Generation: Tasks and Examples
- Decoding Strategies
 - Maximization
 - Sampling

Recap

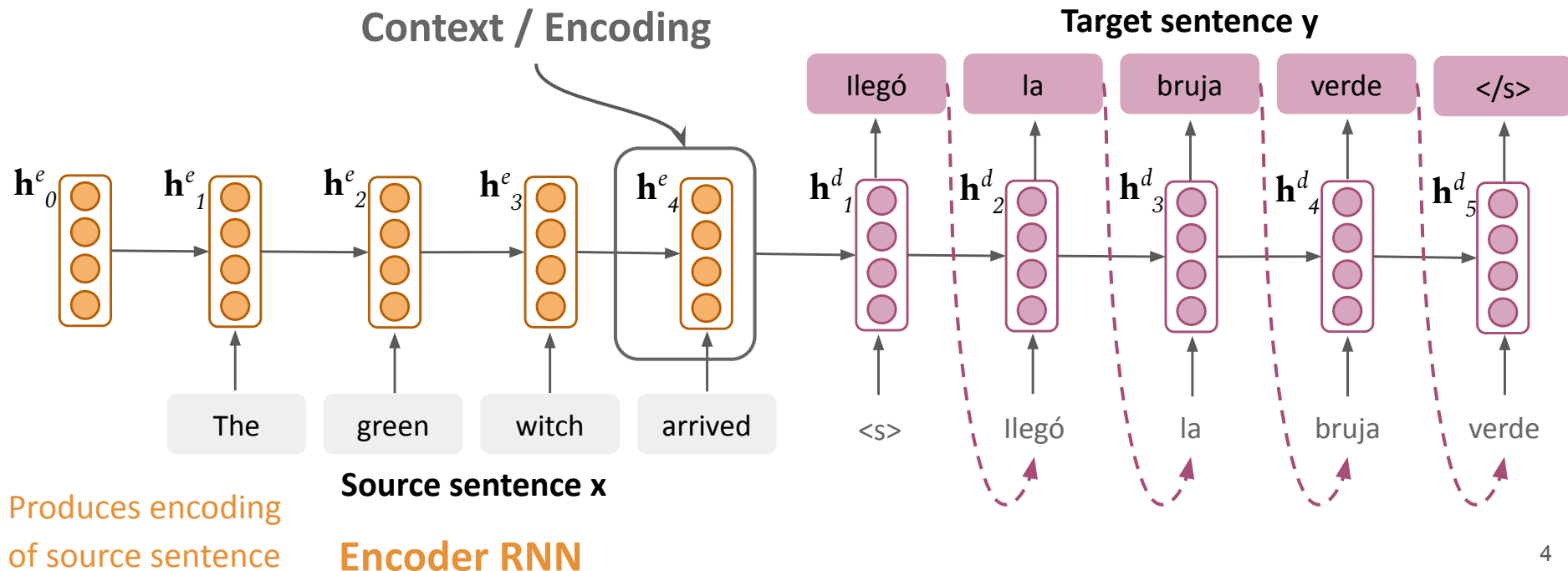
Encoder-Decoder Network

Represents input sequence.

Provides initial hidden state for Decoder RNN

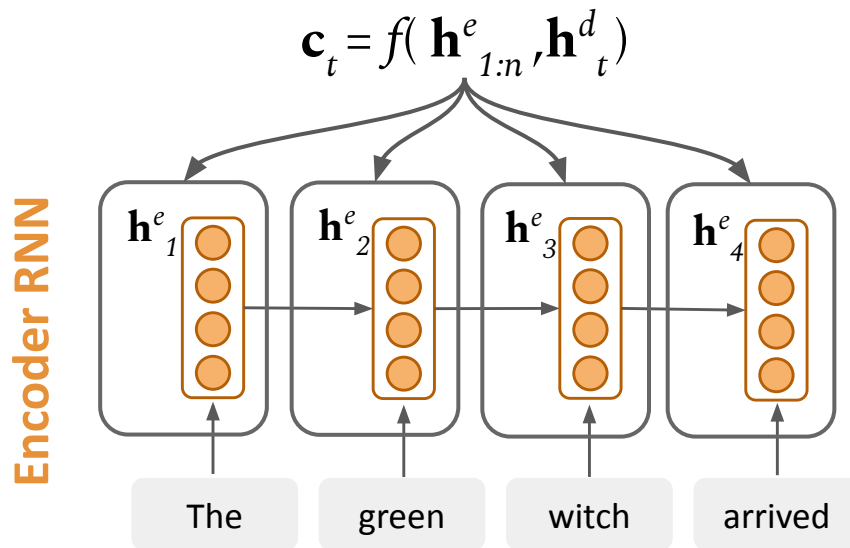
Language Model that produces the target sentence conditioned on the encoding

Decoder RNN

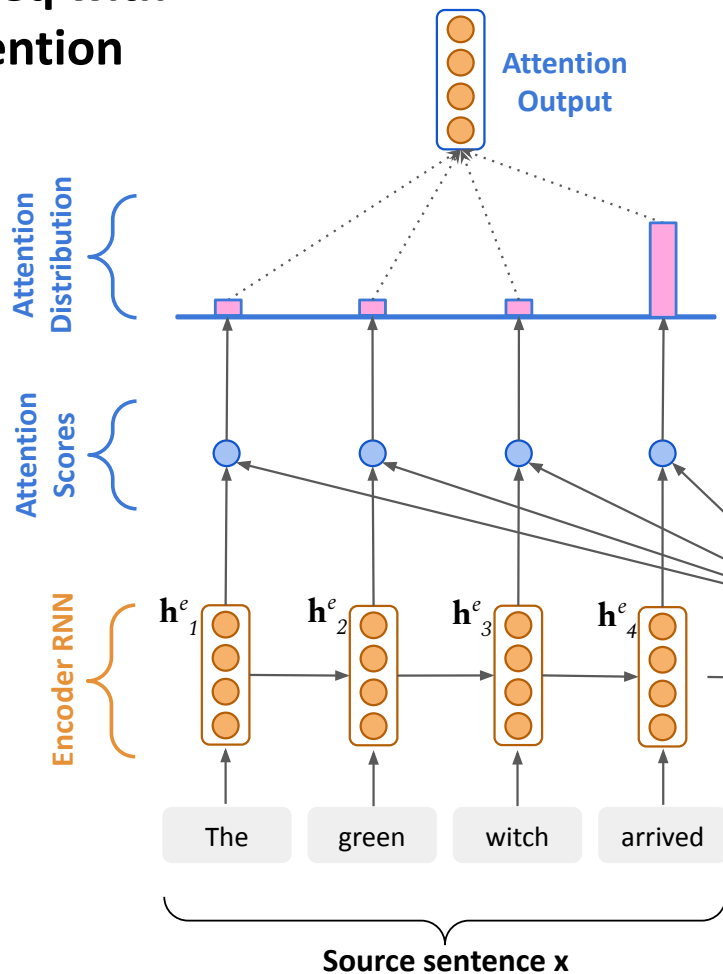


Attention Mechanism

- Attention allows the decoder to focus on a particular part of the source sequence at each time step
- Attention context vector $\mathbf{c}_t$
 - Weighted sum of the encoder hidden states
 - A different context vector at each decoder timestep
 - Weights emphasize source information relevant to the next output token (i.e. **attend** to the relevant information)



Seq2seq with Attention



$$\mathbf{c}_t = \sum_{j=1}^n \alpha_{t,j} \mathbf{h}_j^e$$

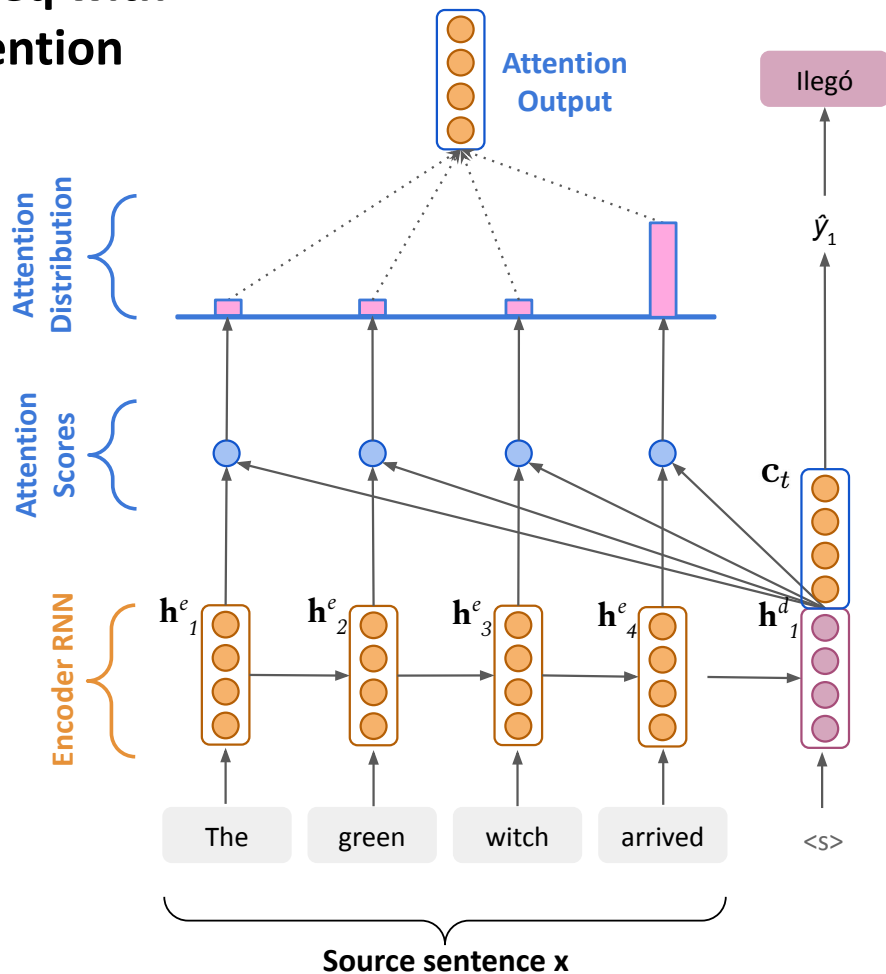
$$\boldsymbol{\alpha}_t = \text{softmax}(\mathbf{s}_t) \in \mathbb{R}^n$$

$$\alpha_{t,j} = \frac{\exp(s_{t,j})}{\sum_{k=1}^n \exp(s_{t,k})} \quad \alpha_{t,j} = \frac{\exp((\mathbf{h}_t^d)^\top \mathbf{h}_j^e)}{\sum_{k=1}^n \exp((\mathbf{h}_t^d)^\top \mathbf{h}_k^e)}$$

$$s_{t,j} = \text{score}(\mathbf{h}_t^d, \mathbf{h}_j^e) = (\mathbf{h}_t^d)^\top \mathbf{h}_j^e, \quad j = 1, \dots, n$$

Decoder RNN

Seq2seq with Attention



$$\hat{y}_t = \text{softmax}(\mathbf{z}_t)$$

$$\mathbf{z}_t = U \begin{bmatrix} \mathbf{h}_t^d \\ \mathbf{c}_t \end{bmatrix} + \mathbf{b}_2$$

$$U \in \mathbb{R}^{|V| \times 2h}, \quad \mathbf{b}_2 \in \mathbb{R}^{|V|}, \quad \mathbf{z}_t \in \mathbb{R}^{|V|}$$

Decoder RNN

Natural Language Generation: Tasks & Examples

Natural Language Generation (NLG)

- **Natural Language Generation (NLG)** is the workhorse of many classic and novel applications
 - AI Assistants, like ChatGPT
 - Translators and Summarizers
- NLG and Natural language understanding (NLU) are two sides of
- the same coin
 - **Natural language understanding:** Learning representations that perform well on downstream tasks
 - To generate good language, one needs to understand language
 - If you understand language, you should be able to generate it (with some effort)

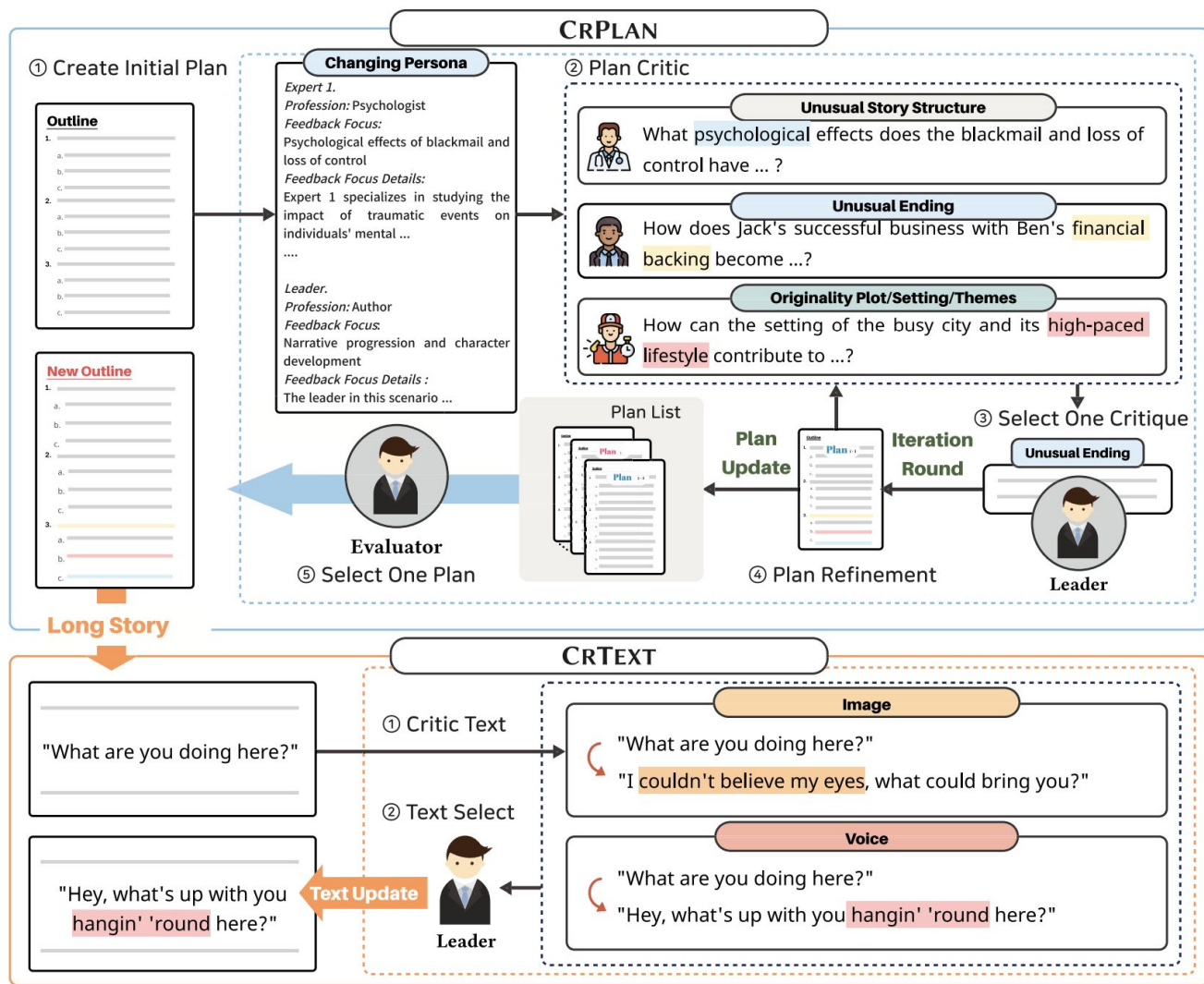
NLG: Tasks and Use Cases

NLG today is much more than “write a sentence”!

- Answer questions using retrieved documents (Retrieval-Augmented Generation)
- Write and edit code
- Call tools / APIs (Generate text & actions)
- Translate speech to text across languages (e.g Spanish speech → English text)
- Generate long reports, plans, and stories
- Produce synthetic training data

Creative Story Generation

Collective Critics for Creative Story Generation, Bae & Kim, EMNLP 2024



Creative Story Generation

[Collective Critics for Creative Story Generation, Bae & Kim, EMNLP 2024](#)

Initial Plan

1. Aimee Kincaid goes home after a long day at work only to find her apartment empty and her loneliness amplified by the silence.

Characters: Aimee Kincaid, Kyle Johnson

a. Aimee Kincaid comes home to her empty apartment after a long day at work ...

Critiques

Critique: What if Aimee's loneliness is personified as a physical entity that she can interact with?

Why: This critique introduces a unique twist by personifying loneliness and turning it into a tangible character. It adds a fantasy element to the story and deviates from the traditional romance storyline, making it quite different, novel, and innovative.

Refined Plan (Personalized)

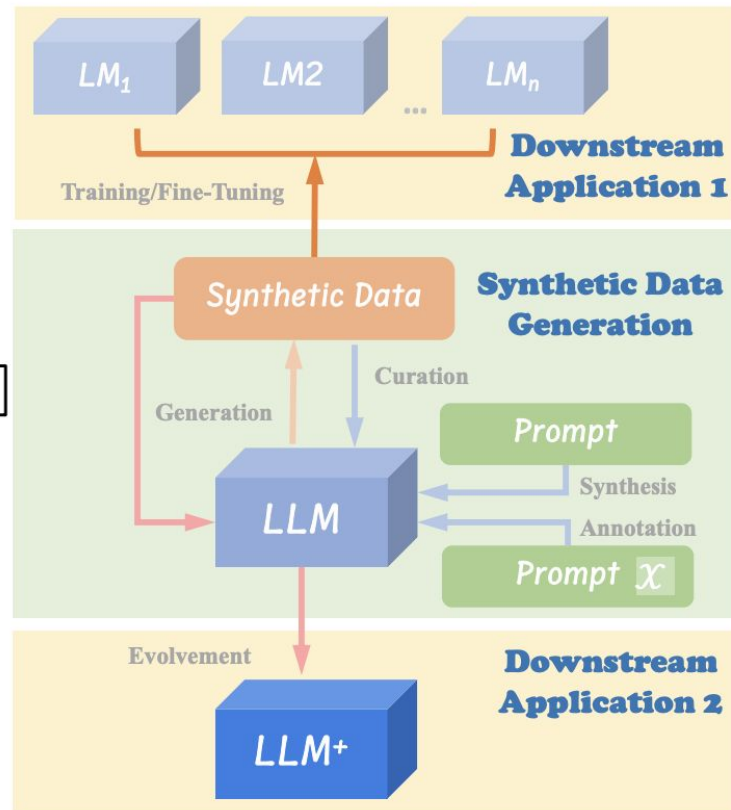
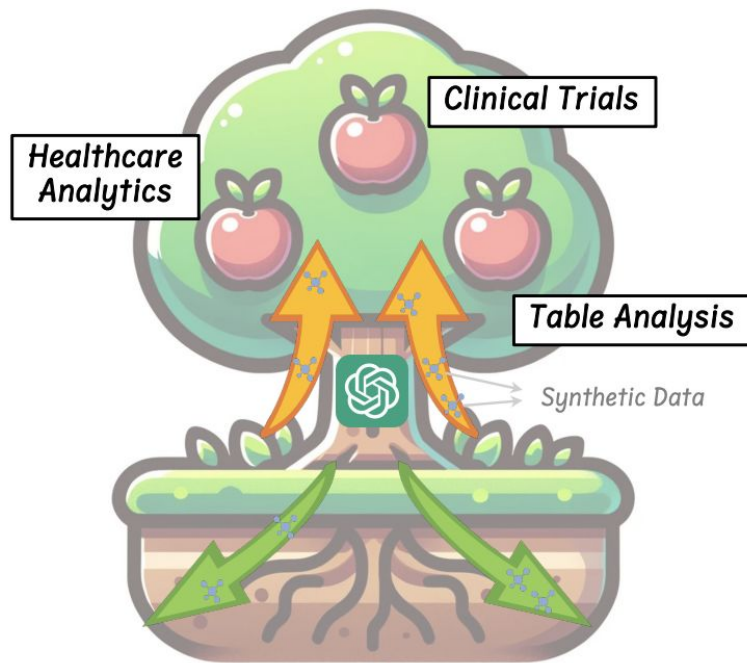
1. Aimee Kincaid goes home after a long day at work only to find her apartment empty and her loneliness manifests as a physical entity ... **Characters: Aimee Kincaid,**

Loneliness(personified)

...

b. **Loneliness interacts with Aimee**, amplifying her feelings of isolation and despair. Scene: Characters: Aimee Kincaid, Loneliness(personified) ...

Synthetic Data Generation



Answer Questions Using Retrieved Documents

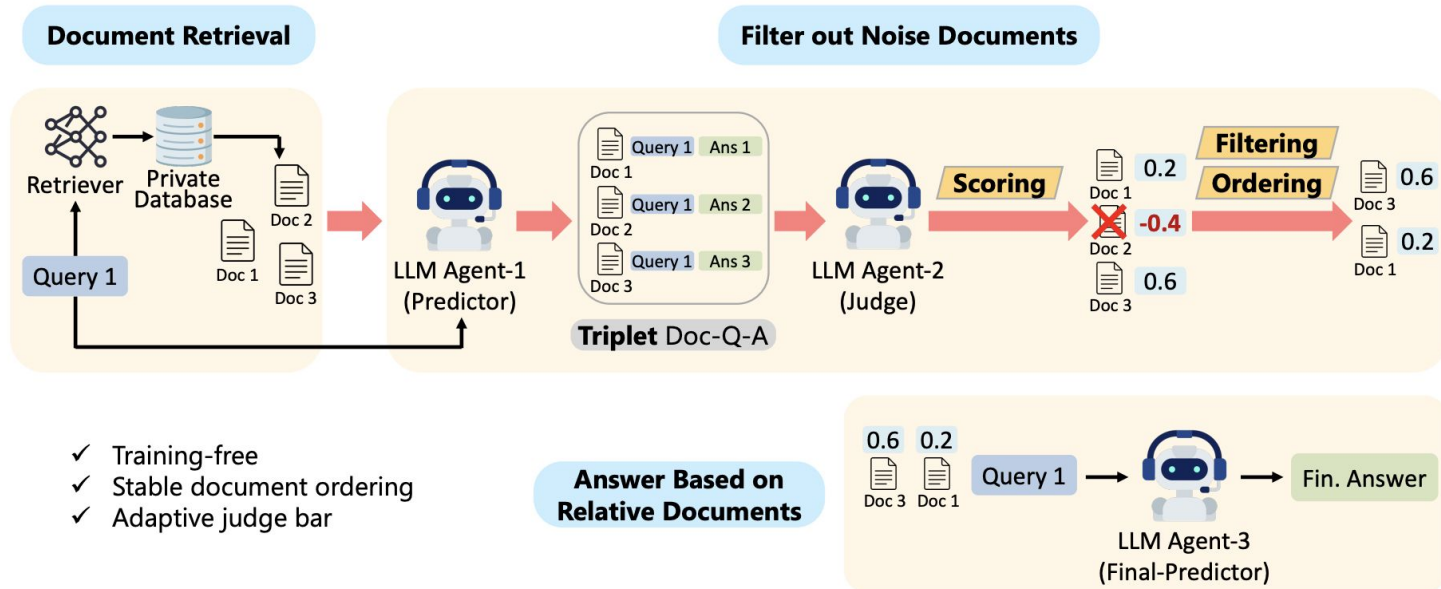


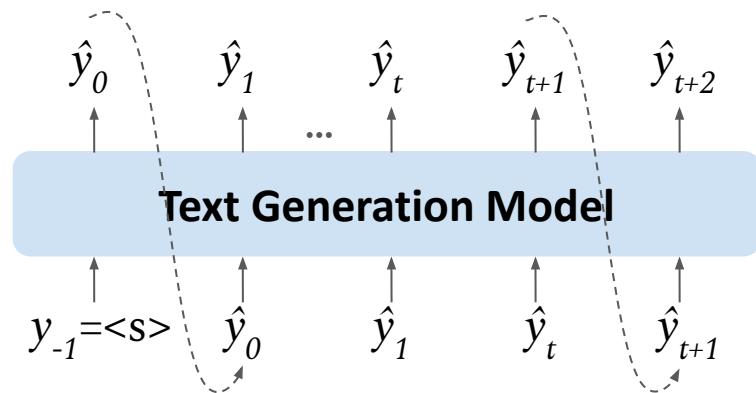
Figure 1: An overview of the proposed framework MAIN-RAG, consisting of three LLM agents to identify noisy retrieved documents for filtering (see Section 3.1). After the retrieval, Agent-1 "Predictor" infers answers for each query; then, Agent-2 "Judge" takes *Doc-Q-A Triplet* to judge if a document is supportive for LLMs to answer the query. "Judge" provides relevant scores for each document for filtering and ordering. Finally, Agent-3 "Final-Predictor" answers the query with the given document list.

Natural Language Generation: Fundamentals

Language Generation: Fundamentals

In autoregressive text generation models, at each time step t , our model takes in a sequence of tokens as input $S = f_\theta(y_{<t}) \in \mathbb{R}^V$ and outputs a new token, $\hat{y}_t$

For model $f_\theta(\cdot)$ and vocabulary V , we get scores $S = f_\theta(y_{<t}) \in \mathbb{R}^V$



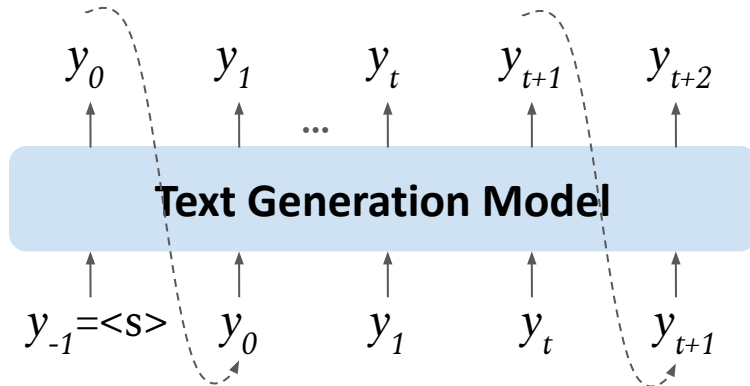
$$P(w \mid y_{<t}) = \frac{\exp(S_w)}{\sum_{v \in \mathcal{V}} \exp(S_v)}$$

Language Generation: Training

- Trained one token at a time to maximize the probability of the next token y_t given the preceding words $y_{<t}$

$$\mathcal{L} = - \sum_{t=1}^T \log P(y_t \mid y_{<t}) = - \sum_{t=1}^T \log \frac{\exp(S_{y_t|y_{<t}})}{\sum_{v \in \mathcal{V}} \exp(S_{v|y_{<t}})}$$

- Classification task at each time step: predict the actual word y_t in training data
- “Teacher forcing” (reset at each time step to the ground truth)



Teacher Forcing

- Strategy for **training** decoders / language models
- At each time step t in decoding we force the system to use the gold target token from training as the next input x_{t+1} , rather than allowing it to rely on the (possibly erroneous) decoder output $\hat{y}_t$
- Runs the risk of **exposure bias!**
 - During training, our model's inputs are gold context tokens from real, human-generated texts
 - At generation time, our model's inputs are previously-decoded tokens

Language Generation: Inference

- At inference, our decoding algorithm defines a function to select a token from the output distribution:

$$\hat{y}_t = g(P(y_t \mid y_{<t}))$$


Decoding Algorithm

- The “obvious” decoding algorithm is to greedily choose the highest probability next token according to the model at each time step:

$$g = \arg \max$$

$$\hat{y}_t = \arg \max_{w \in \mathcal{V}} P(y_t = w \mid y_{<t})$$

Decoding: Maximization

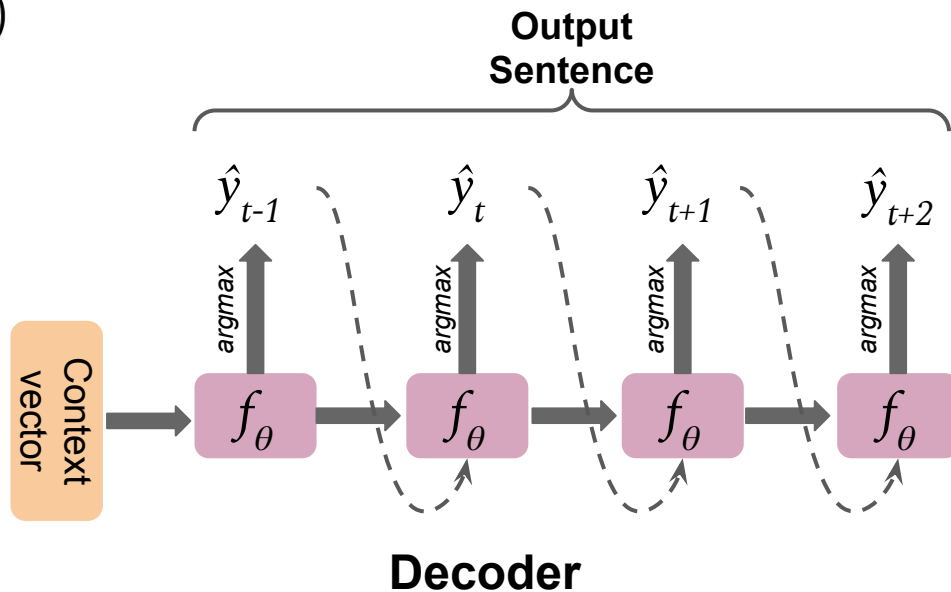
Decoding

- Generation from a language model is also called decoding
 - Think encoder-decoder
 - Also called inference
- Strategy so far: Take $\arg \max$ on each step of the decoder to produce the most probable word on each step
- This is called **greedy decoding**
 - Greedy Strategy: we are not looking ahead, we are making the hastiest decision given all the information we have

Greedy Decoding: Argmax Decoding

- **Decoding function $g(.)$** = select highest probability token using argmax of probability distribution (most probable token at each time step).

$$\hat{y}_t = \arg \max_{w \in \mathcal{V}} P(y_t = w \mid y_{<t})$$

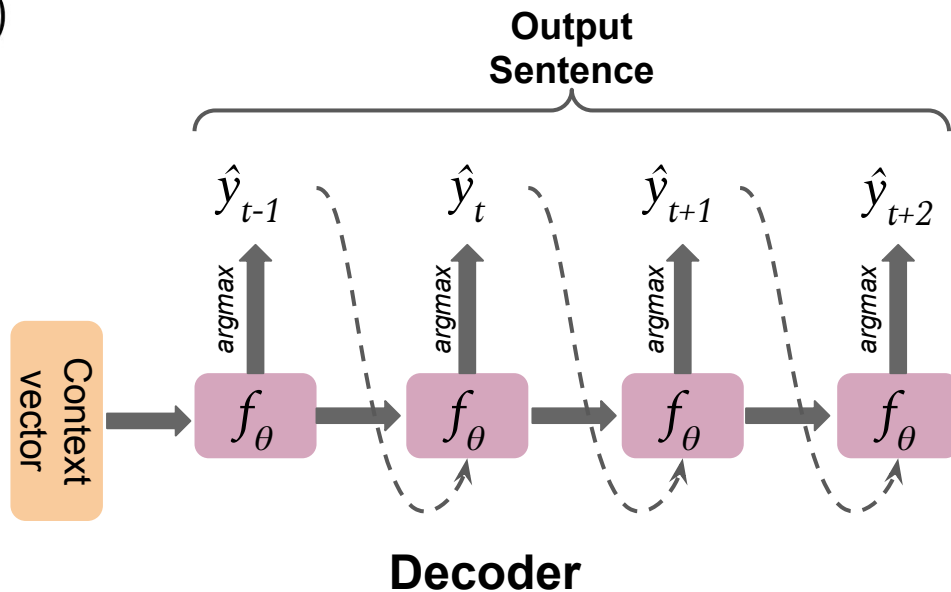
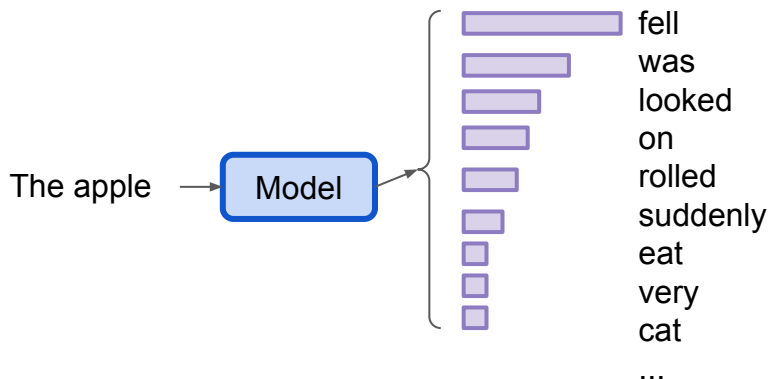


Greedy Decoding: Argmax Decoding

- **Decoding function $g(.)$** = select highest probability token using argmax of probability distribution (most probable token at each time step).

$$\hat{y}_t = \arg \max_{w \in \mathcal{V}} P(y_t = w \mid y_{<t})$$

Target: The apple fell far from the tree.

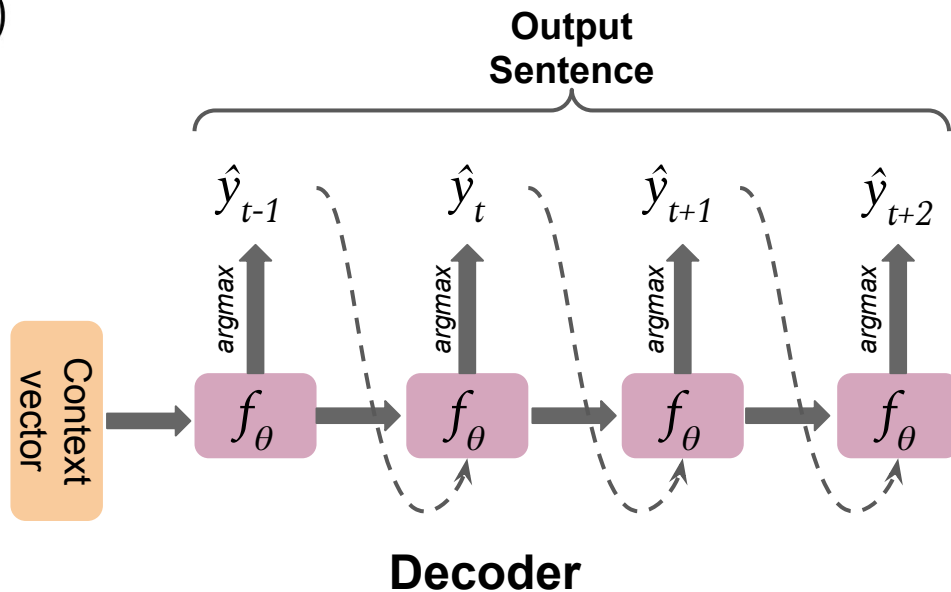
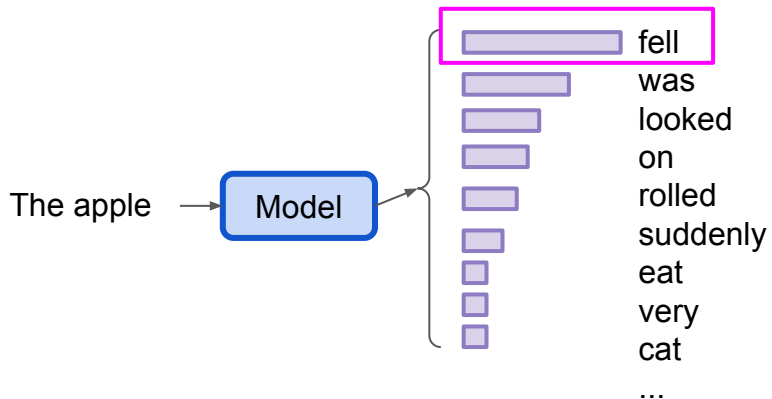


Greedy Decoding: Argmax Decoding

- **Decoding function $g(.)$** = select highest probability token using argmax of probability distribution (most probable token at each time step).

$$\hat{y}_t = \arg \max_{w \in \mathcal{V}} P(y_t = w \mid y_{<t})$$

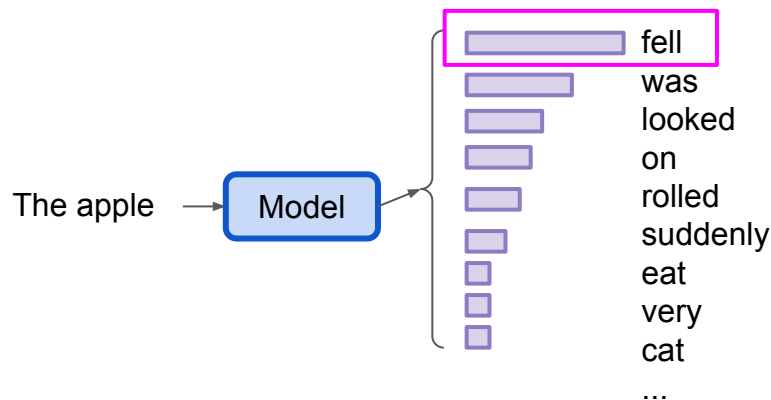
Target: The apple fell far from the tree.



Problems: Argmax Decoding

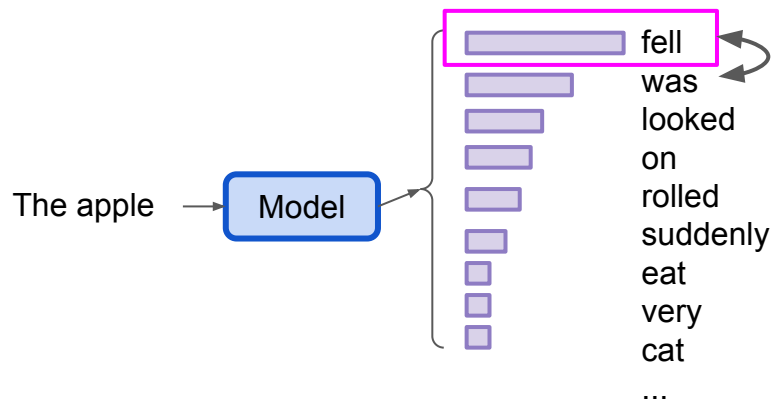
Ideally, we'd like the highest probability token to be the correct target token...

Target: The apple fell far from the tree.



Problems: Argmax Decoding

Target: The apple fell far from the tree.

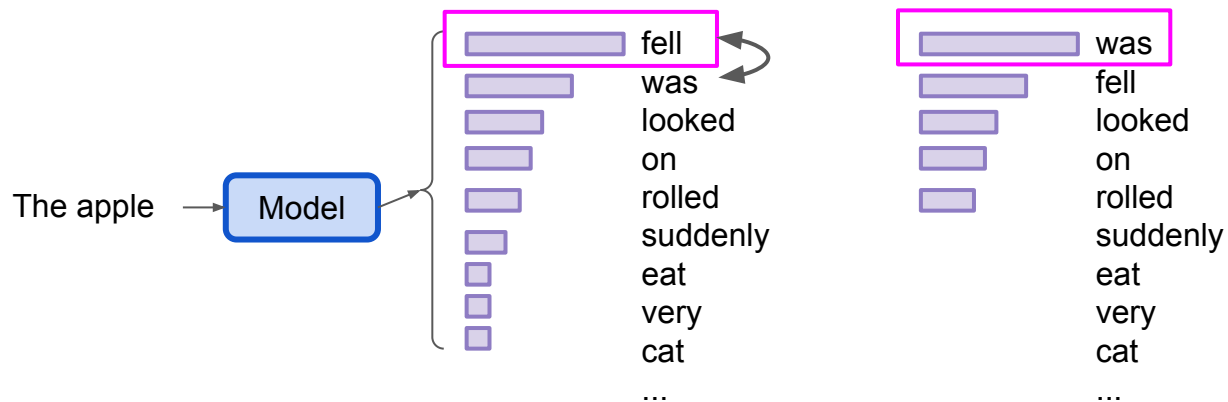


Ideally, we'd like the highest probability token to be the correct target token...

But what if it's not?

Problems: Argmax Decoding

Target: The apple fell far from the tree.



Exhaustive Search

- Ideally, we want to find a (length T) translation y that maximizes

$$\begin{aligned} p_{\theta}(y \mid x) &= p_{\theta}(y_1 \mid x) p_{\theta}(y_2 \mid y_1, x) p_{\theta}(y_3 \mid y_1, y_2, x) \cdots p_{\theta}(y_T \mid y_{<T}, x) \\ &= \prod_{t=1}^T p_{\theta}(y_t \mid y_{<t}, x). \end{aligned}$$

- We could try computing all possible sequences y
- This means that on each step of the decoder, we will be tracking V^T possible partial translations, where V is the vocab size
- This $O(V^T)$ complexity is far too expensive!

Can't Consider All Paths:

Let's find a good, but not guaranteed optimal, solution.

Beam Search Decoding

- Core idea: On each step of decoder, keep track of the k most probable partial sequences (which we call hypotheses)
 - k is the beam size (in practice around 5 to 10, in NMT)
- A hypothesis has a score which is its log probability:

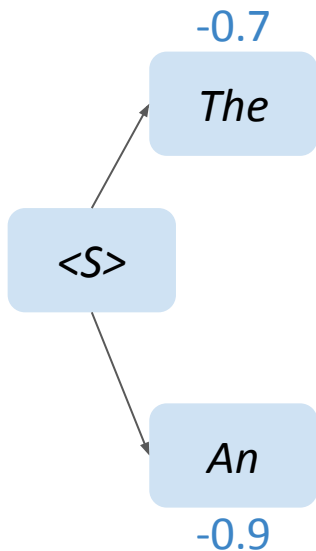
$$\text{score}(y_{1:t}) = \log p_{\theta}(y_{1:t} \mid x) = \sum_{i=1}^t \log p_{\theta}(y_i \mid y_{<i}, x)$$

- Scores are all negative, and higher score is better
 - We search for high-scoring hypotheses, tracking top k on each step
- Beam search is not guaranteed to find optimal solution
- But much more efficient than exhaustive search!

Beam Search Decoding: Example

Beam size: $k = 2$

Blue numbers: $\text{score}(y_{1:t}) = \sum_{i=1}^t \log p_{\theta}(y_i \mid y_{<i}, x)$



Beam Search Decoding: Stopping Criterion

- Greedy Decoding is done until the model produces an $\langle /s \rangle$ token
 - For e.g. $\langle s \rangle$ the apple fell far $\langle /s \rangle$
- In Beam Search Decoding, different hypotheses may produce $\langle /s \rangle$ tokens at different time steps
 - When a hypothesis produces $\langle /s \rangle$, that hypothesis is complete
 - Place it aside and continue exploring other hypotheses via beam search
- Usually we continue beam search until:
 - We reach time step T (where T is some predefined cutoff), or
 - We have at least completed n hypotheses (where n is a predefined cutoff)

Beam Search Decoding

- We have our list of completed hypotheses. Now how to select top one?
- Each hypothesis $y_1, \dots, y_t$ on our list has a score

$$\text{score}(y_{1:t}) = \log p_{\theta}(y_{1:t} \mid x) = \sum_{i=1}^t \log p_{\theta}(y_i \mid y_{<i}, x)$$

Maximization Based Decoding

- Greedy or beam search
- Beam search can be more effective with larger beams
- **But:** generation can be bland or repetitive (i.e. degenerate)

Context: In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

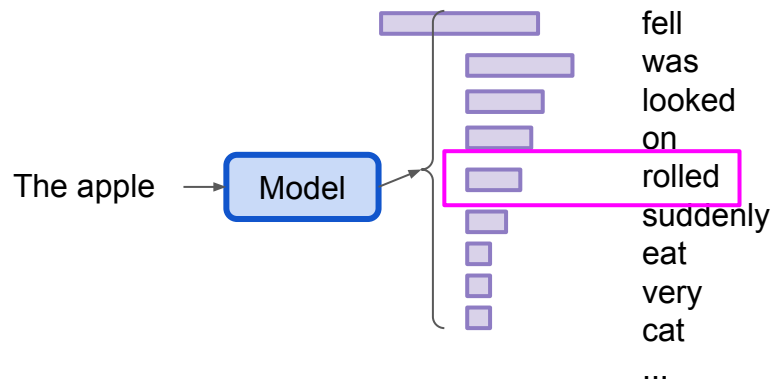
Beam Search, $b=32$:

"The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the Universidad Nacional Autónoma de México (UNAM) and the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de ..."

Decoding: Sampling

Pure Sampling

- **Decoding function $g(.)$** = sample a token from the probability distribut $P_t(w) := p_\theta(y_t = w \mid \hat{y}_{<t}, x) = \frac{\exp(S_w)}{\sum_{v \in \mathcal{V}} \exp(S_v)}$
 - Sample directly from P_t
- Still has access to the entire vocabulary
- But if the model distributions are of low quality, generations will be of low quality as well
- Often results in ill-formed generations
 - No guarantee of fluency



Beam Search vs. Pure Sampling

Context: In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

Beam Search, $b=32$:

"The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the Universidad Nacional Autónoma de México (UNAM) and the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de ..."

Pure Sampling:

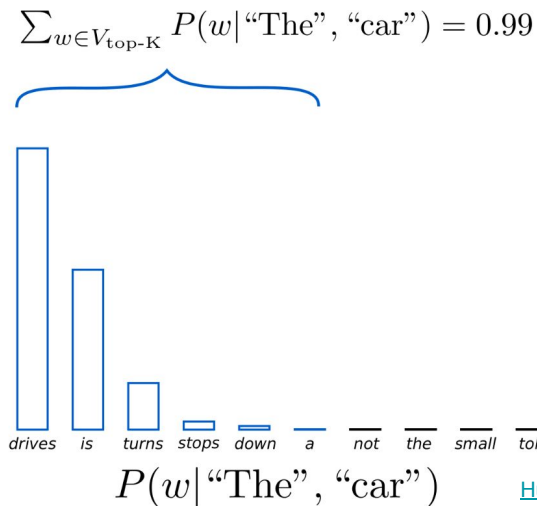
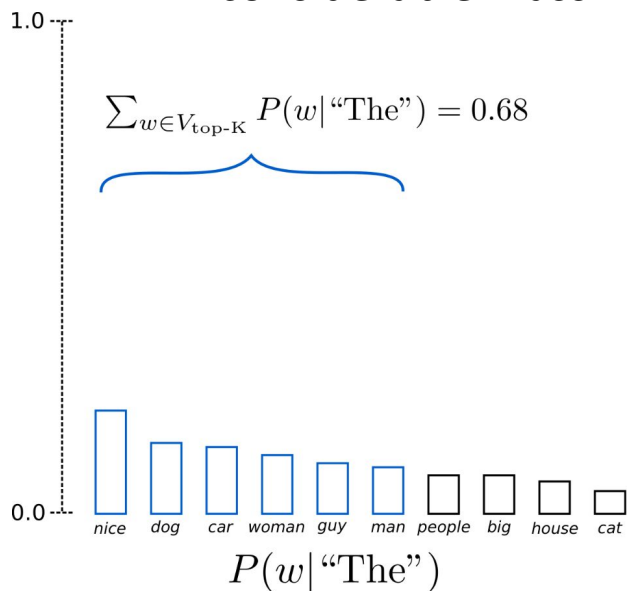
They were cattle called **Bolivian Cavalleros**; they live in a remote desert **uninterrupted** by town, and they speak **huge, beautiful, paradisiacal** Bolivian linguistic thing. They say, 'Lunch, marge.' They don't tell what the lunch is," director Professor Chuperas Omwell told Sky News. "**They've only been talking to scientists, like we're being interviewed by TV reporters. We don't even stick around to be interviewed by TV reporters. Maybe that's how they figured out that they're cosplaying as the Bolivian Cavalleros.**"

Holtzman et al., 2020

Issue: Too much randomness!

Top-K Sampling

- Problem: Pure sampling makes every token in the vocabulary an option
 - Even if most of the probability mass in the distribution is over a limited set of options, the tail of the distribution could be very long and in aggregate have considerable mass



Solution:

Top-K sampling

- Only sample from the top K tokens in the distribution

Top-K Sampling

- Rather than sample from entire distribution P_t , sample only from top- K most probable tokens.
 - Increase K yields more diverse, but risky outputs
 - Decrease K yields more safe but generic outputs
- $K = 1$ (argmax)
- $K = V$ (pure sampling)

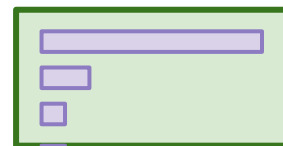
$K=3$



$t = 1$



$t = 2$

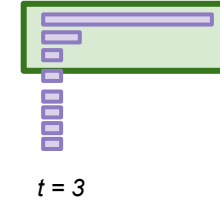
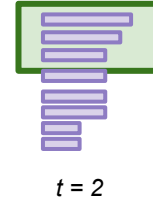
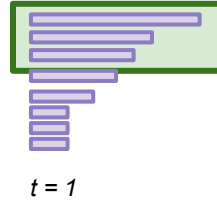


$t = 3$

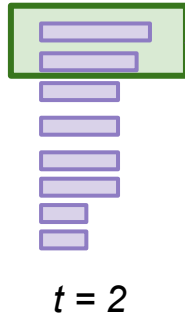
Problems: Top-K Sampling

- How to select K ?

$K=3$

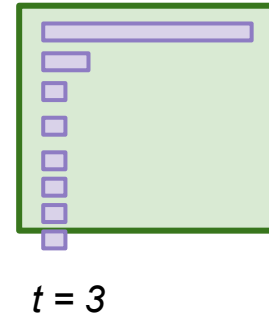


$K=2$



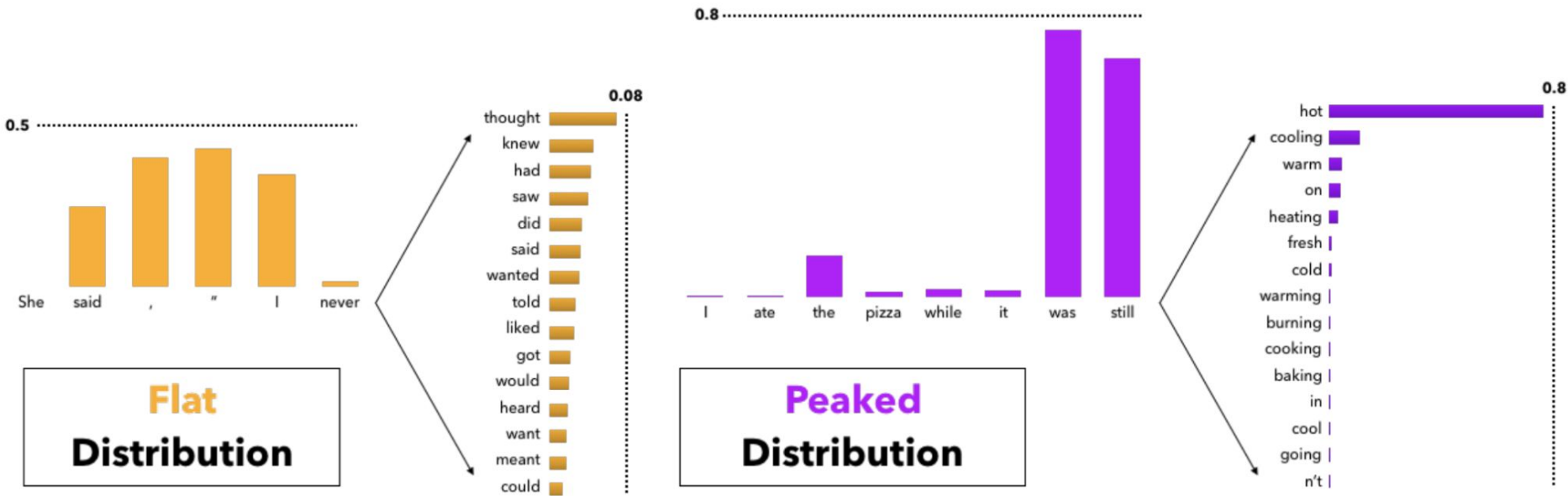
K cutoff too quickly: may not produce probable yet diverse outputs

$K=6$



K cutoff too slowly: may consider irrelevant/invalid tokens and produce nonsense

Concrete Example With Fixed K



[Holtzman et al., 2020](#)

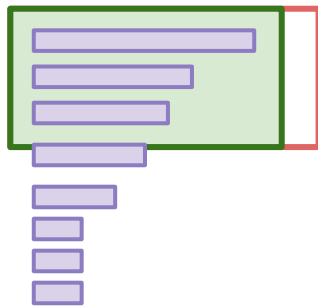
- **Problem:** The probability distributions we sample from are dynamic
 - When the distribution P_t is flatter, a limited K removes many viable options
 - When the distribution P_t is peakier, a high K allows for too many options to have a chance of being selected

Top- p Sampling

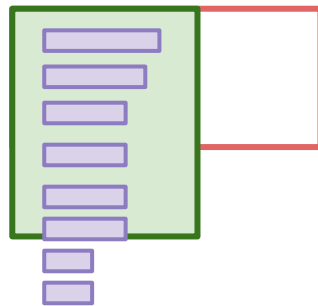
- Solution: sample from where probability mass is concentrated: **nucleus/top- P sampling**
- Sample from all tokens in the top P cumulative probability mass (i.e., where mass is concentrated)
 - Varies K depending on the uniformity of P_t
 - Nucleus is dynamically changing: add probability of each word from softmax until desired probability density of nucleus

$P^*=0.75$

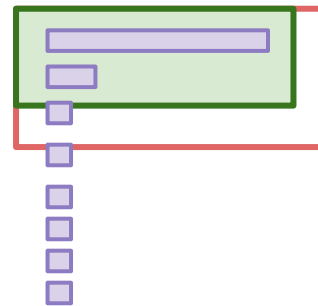
$K=3$



$t = 1$



$t = 2$



$t = 3$

Softmax Temperature Scaling

- We can add a temperature parameter to the softmax function to rebalance P_t

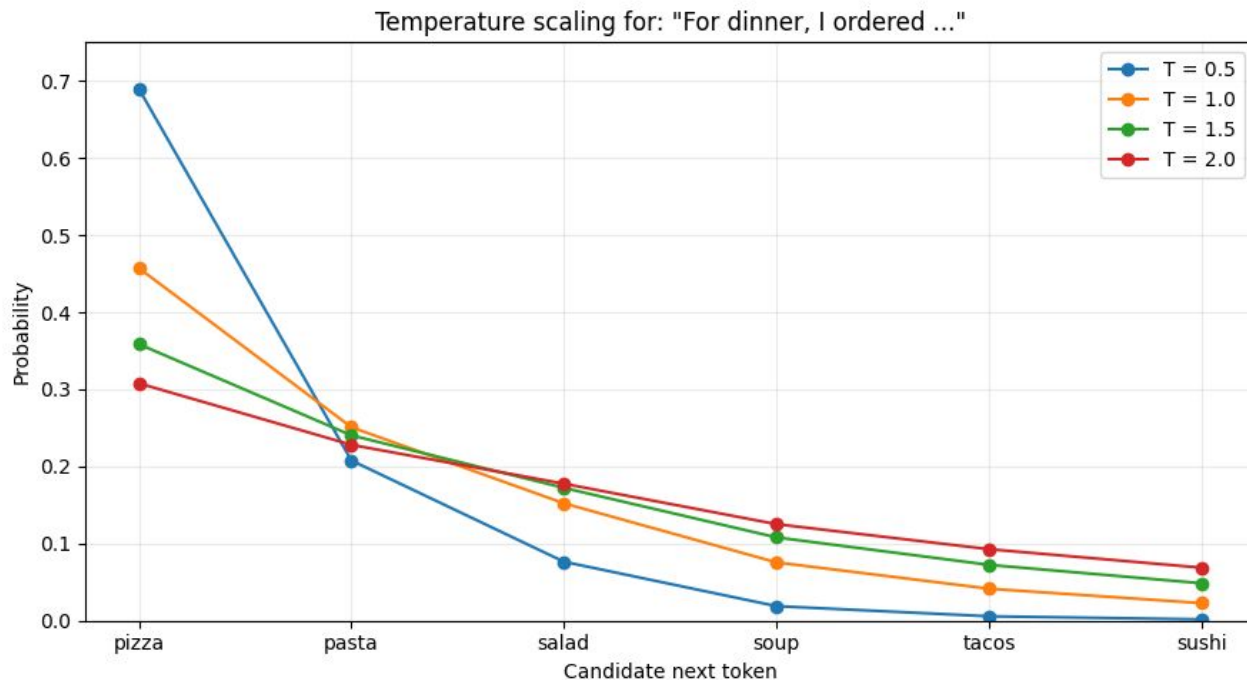
$$p_{\theta}(y_t = w) = \frac{\exp(S_w)}{\sum_{v \in \mathcal{V}} \exp(S_v)} \quad \longrightarrow \quad p_{\theta}(y_t = w) = \frac{\exp(S_w/\tau)}{\sum_{v \in \mathcal{V}} \exp(S_v/\tau)}$$

- **Raise** the temperature τ ($\tau > 1$): flatter, more uniform distribution
 - More diverse: probability spread over vocabulary
 - Potentially beyond random sampling
- **Lower** the temperature τ ($\tau < 1$): spikier, more “peaky” distribution
 - Less diverse: probability concentrated on top words
 - Closer to argmax

Temperature Scaling: Example

$$p_{\theta}(y_t = w) = \frac{\exp(S_w/\tau)}{\sum_{v \in \mathcal{V}} \exp(S_v/\tau)}$$

Token	S_w
pizza	2.2
pasta	1.6
salad	1.1
soup	0.4
tacos	-0.2
sushi	-0.8



TODOs:

- **Homework 2 Due: 10/23**
- **Quiz 2: 10/14**

Next Class: Transformers (I)